

Vibe Coding을 위한 프롬프트 엔지니어링

1. 프롬프트 엔지니어링 개요

❖프롬프트 엔지니어링이란?

- AI 언어 모델에게 원하는 결과를 얻기 위해 프롬프트(입력 문구)를 체계적으로 설계하고 최적화하는 과정
- 프롬프트는 단순한 질문 문장이 아니라, 모델이 어떤 방식으로 사고하고 응답해야 할지를 유도하는 일종의 설계된 지시어

❖프롬프트 엔지니어링의 목적

- 정확성 향상: 모델이 더 올바른 답변을 생성하도록 유도
- 효율성 확보: 한 번의 요청으로 더 나은 결과를 얻어 불필요한 반복 감소
- 일관성 유지: 동일한 질문에 대해 동일한 품질의 응답을 생성
- 자동화 지원: 코드 생성, 데이터 처리, 문서화 작업 등에 활용

2. 프롬프트의 핵심 구성 요소

- ❖ 목표(Task)
  - 무엇을, 왜 하는지 한 줄로 명확히.
- ❖ 입력(Context)
  - 배경, 제약, 참고자료(요약·발췌) 제공.
- ❖ 출력 형식(Output Format)
  - JSON/표/단계 목록 등 구체적 포맷 지정.
- ❖ 지시어(Instructions)
  - 해야 할 것·하지 말아야 할 것(Do/Don't) 분리.
- ❖ 평가기준(Rubric)
  - 품질 기준·검수 규칙.
- ❖ 역할(Role/Audience/Tone)
  - 대상 사용자, 말투, 전문성 레벨.
- ❖ 예시(Few-shot)
  - 좋은/나쁜 예 1–3개로 패턴 고정.

3. 프롬프트 엔지니어링 원칙 및 전략

❖핵심 원칙

- 명확성(Clarity)
- 입력 분리(Delimiters)
- 출력 고정(Format Lock)
- 단계적 분해(Decomposition)
- 최소 충분 컨텍스트(Minimal Sufficient Context)
- 긍정/부정 지시 병행(Do/Don't)
- 예시 기반(Few-shot Grounding)
- 검수 루프(Quality Loop)
- 도구 사용 선언(Tool-aware Prompting)
- 가정(Assumptions)와 한계(Limits) 표기

❖프롬프트 전략

- CoT(Chain of Thought)
- ReAct(Reasoning - Acting)
- 자기 일관성과 역할 기반 프롬프팅

3.1 기본원칙 - 명확성

❖명확성(Clarity)

- AI가 정확히 무엇을 해야 하는지 이해할 수 있도록 구체적이고 명확한 지시를 제공하는 것

❖명확성의 핵심 요소

- 구체적인 지시 사항
  - 모호한 표현 대신 정확한 요구사항을 명시해야 함
  - 예) "글을 써줘" (X) --〉 "300단어 분량의 마케팅 이메일을 작성해줘" (O)
- 명확한 형식 지정
  - 원하는 출력 형식을 구체적으로 지정해야 함
  - 예) "요약해줘" (X) --〉 "3개의 핵심 포인트로 요약하고, 각 포인트는 한 문장으로 작성해줘"
- Context 제공
  - AI가 작업을 수행하는 데 필요한 배경 정보나 제약 조건을 지정해야 함
  - 예) 대상 독자, 사용 목적, 사용 범위

3.2 기본 원칙 - 입력 분리

❖입력 분리(Delimiters)

- 프롬프트의 서로 다른 부분을 명확히 구분하기 위해 특별한 기호나 태그를 사용하는 기법

❖입력 분리의 목적

- 명확한 구조화
  - 프롬프트의 각 섹션을 시각적으로 구분하여 AI가 지시사항과 데이터를 정확히 구별하도록 함
- 혼동 방지
  - 긴 텍스트나 복잡한 데이터가 포함된 경우 유용함
  - 어디까지가 지시사항이고 어디부터가 처리할 데이터인지 명확히 구별하도록 함
- 보안 강화
  - 프롬프트 인젝션 공격을 방지
  - 의도하지 않은 명령 실행 방지

❖주요 구분자

- 삼중 따옴표
- XML 태그
- 마크다운 코드 블록

3.2 기본 원칙 - 입력 분리

❖입력 분리 예시 1 : 삼중따옴표

```
다음 요구사항에 맞는 사용자 인증 시스템을 Node.js와 Express로 개발해주세요:

"""
기능 요구사항:
- 이메일/패스워드 기반 회원가입 및 로그인
- JWT 토큰을 이용한 인증
- 비밀번호는 bcrypt로 암호화
- 이메일 중복 검증
- 로그인 실패 시 적절한 에러 메시지 반환
- 토큰 만료 시간: 24시간

기술 스택:
- Node.js + Express
- MongoDB + Mongoose
- bcryptjs
- jsonwebtoken
"""
```

3.2 기본 원칙 - 입력 분리

❖입력 분리 예시 2 : XML 태그 형식

다음 명세에 맞는 간단한 TODO 애플리케이션을 개발해주세요.

<frontend_요구사항>
- React 18 사용
- Todo 추가, 삭제, 완료 표시 기능
- 반응형 디자인 (모바일 친화적)
- 상태 관리는 useState 사용
- 깔끔한 UI/UX
</frontend_요구사항>
<backend_요구사항>
- Express.js 서버
- RESTful API 설계
- 메모리 기반 데이터 저장 (초기 버전)
- CORS 설정 포함
- JSON 응답 형식
</backend_요구사항>
<api_명세>
GET /api/todos - 모든 할일 조회
POST /api/todos - 새 할일 추가
PUT /api/todos/:id - 할일 수정
DELETE /api/todos/:id - 할일 삭제
</api_명세>

3.2 기본 원칙 - 입력 분리

❖입력 분리 예시 3 : 대괄호

다음 명세에 맞는 JavaScript 유틸리티 함수들을 개발해주세요:

[문자열 처리 함수]
- formatPhoneNumber(number): 전화번호를 "010-1234-5678" 형식으로 변환
- extractHashtags(text): 텍스트에서 해시태그 배열로 추출
- generateSlug(title): 제목을 URL 친화적 슬러그로 변환

[날짜 처리 함수]
- getRelativeTime(date): "3분 전", "2시간 전" 형태로 변환
- formatDateRange(startDate, endDate): 날짜 범위를 문자열로 변환
- isWeekend(date): 주말 여부 확인

[배열 처리 함수]
- groupBy(array, key): 객체 배열을 특정 키로 그룹화
- removeDuplicates(array): 중복 제거
- shuffleArray(array): 배열 요소 랜덤 섞기

각 함수는 TypeScript로 작성하고 JSDoc 주석을 포함해주세요.

3.2 기본 원칙 - 입력 분리

❖입력 분리 예시 3 : 마크다운 형식(권장)

다음 명세에 맞는 간단한 TODO 애플리케이션을 개발해주세요.

frontend_요구사항
- React 19 사용
- Todo 추가, 삭제, 완료 표시 기능
- 반응형 디자인 (모바일 친화적)
- 상태 관리는 useReducer 사용

backend_요구사항
- Node.js + Express 서버
- RESTful API 설계
- JSON으로 요청/응답
- CORS 설정 포함

api_명세
- GET /api/todos - 모든 할일 조회
- POST /api/todos - 새 할일 추가
- PUT /api/todos/:id - 할일 수정
- DELETE /api/todos/:id - 할일 삭제

3.3 기본 원칙 - 출력 고정

❖출력 고정(Format Lock)

- AI가 생성하는 응답의 형식과 구조를 일관되게 유지하도록 강제하는 기법

❖출력 고정의 목적

- 일관성 보장
  - 같은 유형의 요청에 대해 항상 동일한 형식으로 응답을 받아 후처리나 파싱을 용이하도록 함
- 자동화 지정
  - 정해진 형식의 출력은 다른 시스템이나 프로그램에서 쉽게 처리할 수 있어 단계별 작업을 서로 연동하여 처리할 때 유용함
- 구조화된 응답
  - 출력 형식을 제한함으로써 일정 수준 이상의 구조화된 응답을 보장받을 수 있음

❖주로 사용하는 출력 형식 : JSON, Table

3.3 기본 원칙 - 출력 고정

❖개발시 사용 예시 1

현재 코드베이스에 대해 코드 리뷰를 진행하고, 코드 리뷰 결과를 다음 형식으로 응답해주세요.

코드 리뷰 결과
전체 평가: [A/B/C/D/F]

주요 이슈
1. [이슈 설명]
2. [이슈 설명]

개선 제안
- [제안 1]
- [제안 2]

장점
- [장점 1]
- [장점 2]

3.3 기본 원칙 - 출력 고정

❖개발시 사용 예시 2

데이터베이스 테이블 설계 결과물을 다음의 형식으로 응답해주세요.

```markdown
### 테이블명: table_name
| 컬럼명 | 데이터타입 | 제약조건                          | 설명         |
|--------|------------|-----------------------------------|--------------|
| id     | INTEGER    | PRIMARY KEY, AUTO_INCREMENT       | 고유 식별자 |
| ...    | ...        | ...                               | ...          |

### 인덱스
- idx_column_name (column_name)

### 관계
- FOREIGN KEY references other_table(id)

**.....(생략)**
```

3.4 기본 원칙 - 단계적 분해

❖단계적 분해(Decomposition)

- 복잡한 문제나 작업을 더 작고 관리하기 쉬운 단위로 나누어 순차적으로 해결하도록 AI에게 지시하는 기법

❖단계적 분해의 목적

- 복잡성 관리
  - 큰 문제를 작은 단위로 나누어 AI가 각 단계에 집중할 수 있도록 하여 오류를 줄이고 품질을 향상시킴
- 추론 과정 명확화
  - AI가 어떤 과정을 거쳐 결론에 도달했는지 단계별로 확인할 수 있어 결과의 신뢰성을 확보할 수 있음
- 중간 검증
  - 각 단계의 결과를 확인하여 오류가 발생한 지점을 파악하고 수정할 수 있음

3.4 기본 원칙 - 단계적 분해

❖기본 패턴1

- 명시적 단계 지정

```
다음 단계에 따라 문제를 해결해주세요:

1단계: 문제 분석 - 주어진 정보를 파악하고 해결해야 할 문제를 명확히 정의
2단계: 해결 방법 탐색 - 가능한 해결 방법들을 나열하고 비교
3단계: 최적 방법 선택 - 가장 적합한 방법을 선택하고 이유 설명
4단계: 실행 계획 수립 - 구체적인 실행 단계를 작성
5단계: 결과 검증 - 해결책이 요구사항을 만족하는지 확인

** 문제점 리스트
- 복잡한 문제상황1
- 복잡한 문제상황2
- ...
```

3.4 기본 원칙 - 단계적 분해

❖기본 패턴2

- 단계별 출력

```
각 단계의 결과를 명확히 구분해서 보여주세요:

## 1단계: 요구사항 분석
[분석 결과]

## 2단계: 기술 스택 선정
[선정 결과와 이유]

## 3단계: 아키텍처 설계
[설계 내용]

## 4단계: 구현 계획
[구현 순서]

## 5단계: 테스트 전략
[테스트 방법]
```

3.4 기본 원칙 - 단계적 분해

❖개발 상황에서의 예시 1

- 코드 개발 과정 분해

현재 설계된 문서를 반영하여 React 커스텀 훅을 개발하는 과정을 다음과 같이 분해해서 진행해주세요.

요구사항
- ShopingBackend 서비스 데이터를 액세스하는 useShopping 훅 개발

1단계: 훅 인터페이스 설계
- 매개변수 정의
- 반환값 타입 정의
- 사용 예시 작성

2단계: 상태 관리 구조 설계
- 필요한 state 변수들 식별
- 초기값 정의
- 상태 전환 로직 설계

3단계: 핵심 로직 구현
- API 호출 로직
- 에러 처리
- 로딩 상태 관리

4단계: 성능 최적화
- 메모이제이션 적용
- 불필요한 재렌더링 방지
- 메모리 누수 방지

5단계: 테스트 코드 작성
- 단위 테스트
- 통합 테스트
- 에지 케이스 테스트

6단계: 문서화
- JSDoc 주석
- 사용법 가이드
- 예제 코드

각 단계의 완성된 코드와 설명을 제공해야 합니다.

3.4 기본 원칙 - 단계적 분해

❖개발 상황에서의 예시 2

- 디버깅 과정 분해

현재 코드베이스중 backend 영역에 대한 다음 성능 이슈를 단계적으로 분석해서 해결해주세요.

문제
- GET /products 엔드포인트로 type=cosmetic 조건으로 조회했을 때 데이터 Fetch가 갑자기 느려짐

1단계: 문제 현상 파악
- 구체적인 증상 분석
- 발생 조건 확인
- 영향 범위 파악

2단계: 성능 측정
- 로딩 시간 측정 방법
- 병목 지점 식별 도구
- 베이스라인 설정

3단계: 원인 분석
- 네트워크 레벨 분석
- 프론트엔드 코드 분석
- 백엔드 API 분석
- 데이터베이스 쿼리 분석

4단계: 해결책 도출
- 각 원인별 해결 방안
- 우선순위 설정
- 예상 효과 분석

5단계: 구현 및 테스트
- 해결책 구현
- 성능 개선 확인
- 부작용 검사

각 단계별로 구체적인 방법과 도구를 제시해주세요.

3.4 기본 원칙 - 단계적 분해

❖개발 상황에서의 예시 3

- 병렬 분해

설계문서(design-backend.md)를 이용하여 작업들을 병렬로 수행할 수 있도록 분해해주세요.

동시 진행 가능한 작업들:
- 작업 그룹 A: [관련 작업들]
- 작업 그룹 B: [관련 작업들]
- 작업 그룹 C: [관련 작업들]

순차 진행 필요한 작업들:
1. 선행 작업
2. 의존성 있는 작업
3. 최종 통합 작업

각 그룹별 담당자와 일정을 제시해주세요.

3.5 기본 원칙 - 최소 충분 컨텍스트

❖최소 충분 컨텍스트(Minimal Sufficient Context)

- AI가 정확한 결과를 도출하는 데 필요한 최소한의 정보만 제공하여 효율성과 정확성을 동시에 확보하는 기법

❖최소 충분 컨텍스트의 목적

- 효율성 극대화
  - 불필요한 정보를 제거하여 AI의 처리 속도를 높이고 토큰 사용량을 최적화함
- 집중도 향상
  - 핵심 정보에만 집중하게 하여 AI가 혼동하지 않고 정확한 답변을 생성하도록 함
- 비용 최적화
  - 토큰 기반 과금 모델에서 불필요한 비용을 절약함.

❖핵심 원칙

- 필수 정보만 포함되도록 : 작업 수행에 반드시 필요한 정보만 선별하여 제공
- 간결하지만 명확하게 : 정보를 압축하되 의미가 손실되지 않도록 명확하게 표현
- 구조화된 정보 제공 : 산발적인 정보보다는 체계적으로 정리된 정보를 제공

3.5 기본 원칙 - 최소 충분 컨텍스트

❖예시 1

- API 최적화

과도하고 불필요한 컨텍스트 예시

우리 회사는 2010년에 설립되어 현재 직원 수 200명의 중견 IT 기업입니다. 주요 사업 분야는 웹 개발, 모바일 앱 개발, 클라우드 솔루션 등이며, 최근 3년간 매출이 꾸준히 증가하고 있습니다.
개발팀은 총 50명이고 프론트엔드 20명, 백엔드 20명, DevOps 10명으로 구성되어 있습니다.
현재 Node.js v16을 사용하고 있으며, 데이터베이스는 PostgreSQL 13.2를 사용합니다. 서버는 AWS EC2 t3.large 인스턴스 5대를 운영 중이고, 로드밸런서는 ALB를 사용하고 있습니다.
최근 사용자가 급증하면서 API 응답시간이 평균 3초에서 7초로 증가했습니다. 특히 사용자 목록 조회 API(/api/users)가 가장 느린 상황입니다.

사용자 목록 조회 API를 최적화해주세요.

최소 충분 컨텍스트

Node.js + PostgreSQL 환경에서 사용자 목록 조회 API 최적화가 필요합니다.

현재 상황:
- API: GET /api/users
- 응답시간: 7초 (목표: 2초 이하)
- 사용자 테이블: 100만 건
- 현재 쿼리: SELECT * FROM users ORDER BY created_at DESC

제약사항:
- 페이지네이션 필수
- 검색 기능 유지 필요

최적화 방안을 제시해주세요.

3.5 기본 원칙 - 최소 충분 컨텍스트

❖예시 2

- 데이터베이스 스키마 설계

과도하고 불필요한 컨텍스트 예시

우리는 새로운 블로그 플랫폼을 개발하고 있습니다. 이 플랫폼은 Medium이나 Tistory와 유사하지만 더 개발자 친화적인 기능들을 제공할 예정입니다. 예를 들어 코드 하이라이팅, 기술 태그 시스템, 개발자 포트폴리오 연동 등이 있습니다.
사용자는 무료 회원과 프리미엄 회원으로 나뉘며, 프리미엄 회원은 월 9,900원을 지불합니다. 광고 시스템도 도입할 예정이고,
나중에는 구독 기능도 추가할 계획입니다. 현재 팀은 풀스택 개발자 3명으로 구성되어 있고, 6개월 내 출시를 목표로 하고 있습니다.

경쟁사 분석 결과 우리만의 차별점이 필요하다고 판단했습니다.
데이터베이스 스키마를 설계해주세요.

최소 충분 컨텍스트

개발자 블로그 플랫폼 DB 스키마 설계

1. 핵심 엔티티:
- 사용자 (회원가입, 프로필)
- 게시글 (제목, 내용, 태그)
- 댓글 (중첩 댓글 지원)
- 태그 (기술 스택 중심)
2. 주요 기능
- 게시글 작성/수정/삭제
- 댓글 및 대댓글
- 태그 기반 분류
- 사용자 팔로우
- 좋아요/북마크
3. 제약사항
- PostgreSQL 사용
- 확장성 고려 (사용자 100만명 목표)

테이블 스키마와 관계를 설계해주세요.

3.6 기본 원칙 - 긍정/부정 지시 병행

❖긍정/부정 지시 병행(Do/Don't)

- AI에게 수행해야 할 작업과 수행하지 말아야 할 작업을 명확히 구분하여 지시함으로써 더 정확하고 안전한 결과를 얻는 기법
- Will / Will Not 이라고도 함

❖긍정/부정 지시 병행의 목적

- 명확한 경계 설정
  - 무엇을 해야 하고 무엇을 하지 말아야 하는지 명확히 구분하여 AI의 혼동 방지
- 품질 보장
  - 좋은 관행과 나쁜 관행을 동시에 제시하여 일정 수준 이상의 품질을 보장
- 위험 회피
  - 잠재적인 오류나 보안 위험을 사전에 차단

3.6 기본 원칙 - 긍정/부정 지시 병행

❖기본 사용 구조

- 명확한 구분 표시

```
### DO (반드시 수행할 것):
- [긍정적 지시사항들]

### DON'T (절대 금지사항):
- [부정적 지시사항들]
```

- 우선 순위 포함

```
### 필수 DO:
- [핵심 요구사항]

### 권장 DO:
- [추가 권장사항]

### 절대 DON'T:
- [절대 금지사항]

### 권장하지 않는 DON'T:
- [권장하지 않는 사항]
```

3.6 기본 원칙 - 긍정/부정 지시 병행

❖예시 1

- 개발 상황에서 코드 작성 Rule/지침 지정

```
Java, Spring Boot를 이용해 Back-end API를 개발할 때 다음 지침을 지켜야 합니다.

### DO (반드시 수행)
- 모든 입력값에 대해 유효성 검증 구현
- try-catch 블록으로 에러 처리
- HTTP 상태 코드를 정확히 설정
- 민감한 정보는 환경변수로 관리
- API 응답할 때 일관되고 구조화된 JSON 형식 사용
- 로깅을 통한 요청/응답 추적
- rate limiting 적용

### DON'T (절대 금지)
- 비밀번호나 API 키를 하드코딩하지 마라
- 사용자 입력을 직접 SQL 쿼리에 삽입하지 마라
- console.log를 프로덕션 코드에 남기지 마라
- 전역 변수 사용 금지
- 에러 메시지에 민감한 시스템 정보 노출 금지
- CORS 설정 없이 배포 금지
- 인증 없는 중요 엔드포인트 생성 금지
```

3.6 기본 원칙 - 긍정/부정 지시 병행

❖예시 2

- 데이터베이스 설계 지침

```
데이터베이스 설계시의 지침:

### DO (필수 수행사항)
- 모든 테이블에 Primary Key 설정
- 외래키 제약조건 명시적 정의
- 인덱스가 필요한 컬럼 식별 및 생성
- NOT NULL 제약조건 적절히 설정
- 데이터 타입 정확히 선택 (VARCHAR 길이 등)
- 테이블/컬럼명은 snake_case 사용
- 생성/수정 시간 컬럼 포함 (created_at, updated_at)

### DON'T (절대 금지사항)
- 테이블명이나 컬럼명에 공백 사용 금지
- TEXT 타입 남용 금지 (적절한 VARCHAR 길이 설정)
- 중복 데이터 저장 (정규화 원칙 위배) 금지
- 예약어를 테이블/컬럼명으로 사용 금지
- 인덱스 없는 JOIN 조건 컬럼 생성 금지
- CASCADE 삭제 무분별 사용 금지
- 백업 계획 없는 운영 DB 변경 금지
```

3.6 기본 원칙 - 긍정/부정 지시 병행

❖예시 3

- React 컴포넌트 개발 지침

```
React 컴포넌트 개발할 때 다음 규칙을 준수합니다.

### DO (모범 사례)
- 함수형 컴포넌트와 Hooks 사용
- PropTypes 또는 TypeScript로 타입 정의
- useState, useEffect 적절히 활용
- 컴포넌트를 작고 재사용 가능하게 설계
- 조건부 렌더링에 논리 연산자, 3항 연산식 활용
- ErrorBoundary 구현

### DON'T (피해야 할 것)
- 컴포넌트 내부에서 직접 DOM 조작 금지
- 배열 인덱스를 key로 사용 금지 (동적 리스트)
- 컴포넌트에서 직접 API 호출 금지 (커스텀 훅 사용)
- 인라인 객체/함수를 props로 전달 금지
- state 직접 변경 금지 (불변성 유지)
- 중첩된 삼항 연산자 과도 사용 금지
```

3.6 기본 원칙 - 긍정/부정 지시 병행

❖예시 4

- 웹 애플리케이션 개발 보안 규칙

```
웹 애플리케이션 보안 구현 시:

### DO (필수 보안 조치)
- 모든 사용자 입력에 대해 서버사이드 검증기능 필수 구현(backend에서)
- JWT 토큰(access_tokem, refresh_token)에 적절한 만료 시간 설정
- 비밀번호는 bcrypt 등으로 해싱처리 필수
- CSRF 토큰 구현
- XSS 방지를 위한 입력값 이스케이프
- 로그인 시도 제한 (rate limiting)
- 정기적인 보안 의존성 업데이트

### DON'T (보안 위험 행위)
- 클라이언트 사이드에만 의존한 검증 금지
- SQL 인젝션 가능한 쿼리 작성 금지
- 민감한 정보가 URL 파라미터에 포함되지 않도록 처리
- 디버그 정보를 프로덕션에 노출 금지
- 기본 관리자 계정 사용 금지
- 취약한 의존성 라이브러리 사용 금지
- CORS를 '*'로 설정 금지
```

3.6 기본 원칙 - 긍정/부정 지시 병행

❖실제 적용시 권장사항

- 상황별 조정
  - 초보자 대상 → DON'T 항목 상세히
  - 경험자 대상 → DO 항목 중심으로
  - 보안 중요 → 보안 관련 DON'T 강화
  - 성능 중요 → 성능 관련 DO 강화
- 지속적 업데이트
  - 정기적 검토를 통해 새로운 모범 사례가 있다면 DO에 반영
  - 발견된 안티 패턴은 DON'T에 추가
  - 기술 트렌드 변화에 따른 조정
  - 팀의 스킬 능력 향상에 따른 조정
- 적절한 균형 유지
  - DO 항목이 너무 많으면 --〉 핵심만 선별하여 정리
  - DON'T 항목이 너무 많으면 --〉 중요한 것만 선택
  - 서로 모순되는 지침 --〉 우선 순위 명시

3.7 기본 원칙 - 예시 기반

❖예시 기반(Few-shot Grounding)

- AI에게 원하는 출력 형태나 품질을 보여주는 구체적인 예시를 1-3개 제공하여 패턴을 학습시키고 일관된 결과를 얻는 기법

❖예시 기반의 목적

- 패턴 학습
  - 텍스트 설명만으로는 전달하기 어려운 미묘한 스타일이나 형식을 구체적인 예시로 보여줌
- 일관성 보장
  - 동일한 작업을 반복할 때 일관된 품질과 형식의 결과를 얻을 수 있음
- 학습 효율성 향상
  - 긴 설명 대신 몇 개의 예시로 AI가 빠르게 패턴을 파악할 수 있도록 함

3.7 기본 원칙 - 예시 기반

❖기본 구조

- zero-shot : 예시 없음

src/calcUtil.js 파일에 배열 내부의 숫자값을 이용해 최소값, 평균값을 구하는 JavaScript 함수를 작성하고, export 해주세요.

- few-shot : 예시 포함

다음 예시들을 참고하여 비슷한 패턴으로 src/calcUtil.js 파일에 최소갑, 평균값을 구하는 JavaScript 함수를 작성하고 export 해주세요.

예시 1:

```javascript
/**
 * 배열에서 최댓값을 찾는 함수
 * @param {number[]} numbers - 숫자 배열
 * @returns {number} 최댓값
 */
function findMax(numbers) {
    if (!Array.isArray(numbers) || numbers.length === 0) {
        throw new Error('유효하지 않은 입력입니다.');
    }
    return Math.max(...numbers);
}
```

3.7 기본 원칙 - 예시 기반

❖사용 예시 1

- API 문서 작성

현재 코드베이스의 REST API의 각 엔드포인트에 대해서 다음 예시를 참조하여 API 문서를 작성하세요.

예시 1
POST /api/users
- Description : 새로운 사용자를 생성합니다.
- Request Header :
```
Content-Type: application/json
Authorization: Bearer {token}
```
- Request Body :
```
{
    "name": "홍길동",
    "email": "hong@example.com",
    "password": "securePassword123"
}
```

- Response(Success)
```
{
    "success": true,
    "data": {
        "id": "user_12345",
        "name": "홍길동",
        "email": "hong@example.com",
        "created_at": "2024-08-21T10:30:00Z"
    },
    "message": "사용자가 성공적으로 생성되었습니다."
}
```
- Response(Error)
```
{
    "success": false,
    "error": {
        "code": "INVALID",
        "message": "이메일 형식이 올바르지 않습니다."
    }
}
```

3.7 기본 원칙 - 예시 기반

❖사용 예시 2

- 프론트엔드 코드 리뷰

현재의 코드베이스의 프론트엔드 코드에 대해 코드 리뷰를 진행합니다. 다음 코드 리뷰 댓글 예시들을 참고하여 동일한 패턴으로 리뷰해주세요.

예시 1
현재 코드
```
// src/components/BoardItem.tsx : Line 14
const result = items.map(item => item.id).filter(id => id > 100);
```
개선방안 제시
```
const result = items.reduce((acc, item) => {
    if (item.id > 100) acc.push(item.id);
    return acc;
}, []);
```
제시 이유
map + filter 체인은 배열을 두 번 순회하지만, reduce는 한 번만 순회하여 성능이 더 좋음
- 우선 순위 : 보통

예시 2
.....(생략)

3.8 기본 원칙 - 검수 루프

❖검수 루프(Quality Loop)

- AI가 생성한 결과물을 반복적 피드백으로 체계적으로 검증하고 수정과정을 통해 결과물의 품질을 보장하도록 하는 기법

❖검수 루프의 적용 목적

- 단계적 품질 향상
  - 한 번에 완벽한 결과를 기대하기보다는 여러 번의 검증과 개선을 통해 품질을 점진적으로 높임
- 체계적 오류 발견
  - 구조화된 검수 과정을 통해 놓치기 쉬운 문제점들을 체계적으로 발견함
- 학습 효과 극대화
  - AI에게 구체적인 피드백을 제공하여 더 나은 결과를 생성하도록 유도함

3.8 기본 원칙 - 검수 루프

❖검수 루프의 순환 과정

- 생성(Generation) – 초기 프롬프트로 AI 결과 생성
- 평가 (Evaluation) – 결과의 명확성, 완성도를 검토, 평가
- 개선(Refinement) – 문제점을 식별해 결과물 수정,보강
- 생성 –〉 평가 –〉 개선 –〉 평가 –〉 개선 –〉 ...
- 이 과정을 여러 차례 반복적으로 진행하여 고품질의 결과물 획득
- **먼저 결과를 생성한 뒤 결과물의 품질과 정확성을 검증하고 개선하는 것이 목적**

❖검수 루프를 위해 필요한 것

- 검수 기준 설정
- 토큰 비용 절감을 위해 캐시 기능 이용 –〉 Claude Code는 토큰 캐싱 기능 내장
- 다양한 도구 사용
  - 정적 분석 도구, 보안 취약점 테스트 도구
  - 단위 테스트 도구, API 테스트 도구
  - 성능 측정 도구, 코드 리뷰 자동화 도구

3.8 기본 원칙 - 검수 루프

❖검수 기준 예시

- 품질 검수 체크 리스트 예시

```
** 품질 검수 체크리스트: **

#### 기능성 (Functionality)
- [ ] 요구사항 충족도
- [ ] 핵심 기능 동작 여부
- [ ] 엣지 케이스 처리

#### 코드 품질 (Code Quality)
- [ ] 가독성 및 명확성
- [ ] 재사용성
- [ ] 유지보수성

#### 성능 (Performance)
- [ ] 실행 속도
- [ ] 메모리 효율성
- [ ] 확장성
```

```
#### 보안 (Security)
- [ ] 입력값 검증
- [ ] 권한 체크
- [ ] 데이터 보호

#### 테스트 (Testing)
- [ ] 테스트 코드 포함
- [ ] 커버리지 적절성
- [ ] 시나리오 완성도

#### 문서화 (Documentation)
- [ ] 주석 적절성
- [ ] API 문서 완성도
- [ ] 사용법 명시
```

3.8 기본 원칙 - 검수 루프

❖개발 상황에서의 예시

- 1단계 : 초기 생성 요청

```
사용자 로그인 API를 Node.js + Express로 개발해주세요.

### 요구사항
- POST /api/auth/login
- 이메일/패스워드 검증
- JWT 토큰 반환
- 에러 처리 포함
```

- 2단계 : AI가 생성한 코드 확인

```
app.post('/api/auth/login', async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });
  ......(생략)
  res.json({ token });
});
```

3.8 기본 원칙 - 검수 루프

❖개발 상황에서의 예시(이어서)

- 3단계 : 생성된 코드 검수 및 피드백

생성된 코드 검수 결과:

발견된 문제점
- 입력값 검증 없음 (email, password 유효성)
- 하드코딩된 JWT 시크릿
- 토큰 만료시간 설정 없음
- 에러 로깅 부재
- 응답 형식 일관성 부족
- Rate Limiting 없음
- 패스워드 BruteForce 공격 대응 없음

개선 요청
- 입력값 유효성 검증 미들웨어 추가
- 환경변수로 시크릿 관리
- 토큰 만료시간 설정
- 로깅 시스템 추가
- 로그인 시도 제한 구현
- 보안 헤더 설정

3.8 기본 원칙 - 검수 루프

❖개발 상황에서의 예시(이어서)

- 4단계 : 개선된 코드
  - 생략
- 5단계 : 다시 검수 및 피드백

```
개선된 코드 재검수

### 해결된 항목
- 입력값 검증 완료
- JWT 시크릿 환경변수화
- 토큰 만료시간 설정
- 로깅 시스템 추가
- Rate limiting 구현
- 일관된 응답 형식

### 추가 개선 가능 항목
- 테스트 코드 추가
- API 문서화
- 헬스체크 엔드포인트
- 모니터링 메트릭

최종 승인하거나 추가 개선을 진행하시겠습니까?
```

3.8 기본 원칙 - 검수 루프

❖데이터베이스 설계시의 예시

- 1단계 : 초기 프롬프트로 생성

```
블로그 플랫폼을 위한 PostgreSQL 데이터베이스 스키마를 설계해주세요.

### 기본 요구사항
- 사용자, 게시글, 댓글 관리
- 태그 시스템
- 좋아요 기능
```

- 2단계 : 첫번째 생성된 결과물 예시

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255),
  name VARCHAR(100)
);

CREATE TABLE posts (
  id SERIAL PRIMARY KEY,
  title VARCHAR(200),
  content TEXT,
  user_id INTEGER REFERENCES users(id)
);
```

```
CREATE TABLE comments (
  id SERIAL PRIMARY KEY,
  content TEXT,
  post_id INTEGER REFERENCES posts(id),
  user_id INTEGER REFERENCES users(id)
);
```

3.8 기본 원칙 - 검수 루프

❖데이터베이스 설계시의 예시

- 3단계 : 검수 및 피드백

데이터베이스 스키마 검수 결과:

발견된 문제점
- 필수 제약조건 누락 (NOT NULL, UNIQUE)
- 인덱스 설정 없음
- 타임스탬프 컬럼 없음
- 태그 시스템 미구현
- 좋아요 기능 테이블 없음
- 비밀번호 저장 컬럼 없음
- 논리 삭제 고려 없음

개선 요청
- 적절한 제약조건 추가
- 성능을 위한 인덱스 설계
- audit 컬럼 추가 (created_at, updated_at)
- 태그 다대다 관계 구현
- 좋아요 시스템 설계
- 사용자 인증 정보 추가
- 논리 삭제 컬럼 추가

3.8 기본 원칙 - 검수 루프

❖데이터베이스 설계시의 예시

- 4단계 : 개선된 스키마

```
-- 사용자 테이블
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    name VARCHAR(100) NOT NULL,
    bio TEXT,
    avatar_url TEXT,
    is_active BOOLEAN DEFAULT true,
    email_verified_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP
);
.....(중략)

-- 인덱스 생성
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_active ON users(is_active) WHERE deleted_at IS NULL;
.....(중략)
```

3.8 기본 원칙 - 검수 루프

❖검수 루프 – 프롬프트 개선 방법

- 이 기법은 프롬프트 자체를 개선하는 방법
  - 프롬프트 자체가 결과물인 경우도 존재
  - 프롬프트 + 결과 둘다 중요한 경우
- 적용 방법
  - 프롬프트 1 –〉 결과 –〉 평가 –〉 프롬프트 수정 –〉 결과 –〉 평가 –〉 프롬프트 수정 –〉 …
- 이 기법을 적용하기 위해 필수적인 것
  - 결과를 다시 이전 시점으로 되돌리기 위한 체크포인트 기능
  - 따라서 git 과 같은 도구의 사용이 필수임
- 특징 및 장단점
  - 프롬프트를 관리하고 향후 재사용해야 할 필요가 있는 경우 사용
  - 초기 토큰 비용이 많이 듬

3.8 기본 원칙 - 검수 루프

❖검수 루프 최적화 전략

- 검수 기준 세분화
  - Level 1 (기본) : 기능 동작 여부 검수
  - Level 2 (표준) : 코드 품질 + 에러 처리
  - Level 3 (고급) : 성능 + 보안 + 테스트
  - Level 4 (전문) : 확장성, 모니터링, 문서화
- 자동화된 검수 실행
  - 자동 스크립트를 활용해 검수 실행
  - AI를 이용해 자동화된 검수 실행
- 반복 중단 조건 지정
  - 품질 기준 달성했을 때 반복 중단
  - 시간/비용 제약
    - 반복을 하더라도 개선 효과가 미미하다면 중단
    - 추가 개선 비용 〉 이익, 효과
  - 주요 이슈 존재 여부 : 주요 이슈가 없다면 중단

3.9 기본 원칙 - 도구 사용 선언

❖도구 사용 선언(Tool-aware Prompting)

- AI가 활용할 수 있는 특정 도구나 기능을 명시적으로 언급하고 어떻게 사용할지 지시하여 더 정확하고 실용적인 결과를 얻는 기법

❖도구 사용 선언의 목적

- 도구 명확성
  - AI가 사용할 수 있는 도구의 존재를 명확히 알려주어 활용하도록 유도함
- 작업 효율성 향상
  - 올바른 도구 선택을 통해 작업 시간을 단축하고 정확도를 높임
- 결과 검증 가능성
  - 실제 도구로 실행 가능한 코드나 명령어를 생성하여 즉시 검증할 수 있습니다.

❖도구에 대한 두가지 관점

- 앱 개발시 사용할 기술 스택 제한
- Claude Code와 같은 AI 에이전트가 사용하는 도구 제한
  - 내장 도구 : Bash, WebFetch, WebSearch, Read, Edit, Create
  - MCP가 제공하는 도구

3.9 기본 원칙 - 도구 사용 선언

❖기본 구조

▪ 도구 명시 패턴 지정

```
다음 도구들을 활용하여 작업해주세요:

### 사용 가능한 도구:
- [도구명 1]: [용도 설명]
- [도구명 2]: [용도 설명]

### 요구사항:
[구체적인 작업 내용]

### 도구 사용 지침:
- [도구별 사용 방법이나 제약사항]
```

▪ 도구별 역할 분담

```
작업을 다음과 같이 도구별로 분담해주세요:

### 분석 단계: [분석 도구]
### 구현 단계: [개발 도구]
### 테스트 단계: [테스트 도구]
### 모니터링 단계: [모니터링 도구]
```

3.9 기본 원칙 - 도구 사용 선언

❖사용 예시 1

- 프론트엔드 개발 상황에서의 프롬프트 예시

```
다음 도구들을 활용해서 React + TypeScript 프로젝트 개발을 진행합니다.

### 작업 요구사항
사용자 등록 폼 컴포넌트 개발

### 개발 도구
- Vite: 빌드 도구 및 개발 서버
- ESLint: 코드 품질 검사
- Prettier: 코드 포맷팅
- Tailwind CSS: 스타일링
- React Hook Form: 폼 관리

### 도구 사용 지침
- Vite 설정에서 TypeScript, React 플러그인 포함
- ESLint에서 React Hooks 규칙 적용
- Tailwind로 반응형 디자인 구현
- React Hook Form으로 폼 상태 관리

각 도구의 설정 파일과 사용 예시를 모두 포함해서 작성해주세요.
```

3.9 기본 원칙 - 도구 사용 선언

❖사용 예시 2

- 데이터베이스 도구 활용

```
PostgreSQL 데이터베이스 최적화 작업을 위해 다음 도구들을 활용해주세요:

### 분석 도구
- pg_stat_statements: 쿼리 성능 통계
- EXPLAIN ANALYZE: 쿼리 실행 계획 분석
- pg_stat_user_tables: 테이블 사용 통계
- pgBadger: 로그 분석

### 모니터링 도구
- pg_stat_activity: 현재 활성 세션
- pg_locks: 락 정보
- pg_stat_database: 데이터베이스 통계

### 작업 요구사항
전자상거래 사이트의 주문 조회 성능 개선

각 도구를 사용한 구체적인 SQL 쿼리와 분석 방법을 제시해주세요.
```

3.9 기본 원칙 - 도구 사용 선언

❖도구 선택 가이드라인

▪ 작업 유형별 도구 매핑

- 프론트엔드 개발
  - 번들러: Vite, Webpack, Rollup
  - 테스팅: Jest, Cypress, Playwright
  - 스타일링: Tailwind, Styled-components, Emotion
  - 상태관리: Redux, Zustand, Jotai
- 백엔드 개발
  - 프레임워크: Express, Fastify, NestJS
  - ORM: Prisma, TypeORM, Sequelize
  - API 문서: Swagger, GraphQL Playground
  - 캐싱: Redis, Memcached
- 데이터베이스
  - RDBMS: PostgreSQL, MySQL, SQLite
  - NoSQL: MongoDB, CouchDB, DynamoDB
  - 마이그레이션: Flyway, Liquibase, Prisma

3.9 기본 원칙 - 도구 사용 선언

❖도구 선택 가이드라인(이어서)

- 도구 조합 최적화 예시
  - Level 1 : 기본 스택
    - 개발 서버: Vite
    - 타입 체킹: TypeScript
    - 린팅: ESLint
    - 포맷팅: Prettier
  - Level 2 : 전문 스택
    - 모노레포: Nx 또는 Lerna
    - 테스팅: Jest + Testing Library
    - CI/CD: GitHub Actions
    - 코드 품질: Husky + lint-staged
    - 문서화: Storybook + TypeDoc
  - Level 3 : 엔터프라이즈 스택
    - 마이크로서비스: Spring Boot + Spring Cloud
    - 데이터베이스: PostgreSQL + Redis
    - 컨테이너: Kubernetes + Helm
    - 모니터링: Prometheus + Grafana + ELK Stack

3.10 기본 원칙 - 가정과 한계 표기

❖가정(Assumption)과 한계(Limit) 표기

- 전제 조건과 제약 사항을 명확히 하여 AI로부터 현실적이고 실용적인 결과를 얻는 기법
- 특히 복잡한 문제를 다루거나 오해 가능성이 높은 상황에서 매우 중요

❖가정과 한계 표기의 목적

- 현실적 결과 보장
  - AI가 실제 환경의 제약사항을 고려하여 실현 가능한 솔루션을 제시하도록 함.
- 오해 방지
  - 이상적인 환경이 아닌 실제 조건에서 작동하는 코드를 생성함.
- 정확성 향상
  - 불필요한 해석의 여지를 줄여 정확성 향상

3.10 기본 원칙 - 가정과 한계 표기

❖사용 예시 1

- React + Node.js 애플리케이션 개발하는 환경

프론트엔드는 Reac, 백엔드는 Node.js를 이용하여 실시간 채팅 애플리케이션을 개발해주세요.

가정사항
- 동시 접속자: 최대 1,000명
- 메시지 보존 기간: 30일
- 사용자는 모던 브라우저 사용 (Chrome, Firefox, Safari 최신 버전)
- 서버는 AWS EC2 t3.medium 인스턴스 (2vCPU, 4GB RAM)
- 데이터베이스: PostgreSQL 12+
- 네트워크: 일반적인 인터넷 연결 (100ms 이하 지연시간)

기술적 한계
- 파일 첨부: 10MB 이하, 이미지만 지원
- 메시지 길이: 1,000자 이하
- 방당 최대 참여자: 50명
- 메시지 검색: 최근 1,000개 메시지만

제약 조건
- 법적 제약: GDPR 준수 필요

💡 가정 변경 시 고려사항도 포함해주세요.

3.10 기본 원칙 - 가정과 한계 표기

❖사용 예시 2

- 데이터베이스 설계 예시

전자상거래 사이트의 주문 관리 시스템 데이터베이스를 설계해주세요.

가정사항
- 일일 주문량: 평균 1,000건, 최대 5,000건 (블랙프라이데이)
- 상품 수: 약 50,000개
- 동시 사용자: 최대 500명
- 데이터 보존: 주문 데이터 7년간 보관 (세법)
- 평균 주문당 상품 개수: 3개
- 결제 방식: 신용카드, 계좌이체, 간편결제

성능,인프라 한계
- 데이터베이스: PostgreSQL on AWS RDS t3.large
- 스토리지: 1TB 제한
- 백업 윈도우: 새벽 2-4시 (2시간)
- 쿼리 타임아웃: 30초

제약 조건
- 다운타임: 월 4시간 이하
- 같은 코드베이스의 legacy 디렉토리에 존재하는 기존 레거시 시스템과 호환성 유지

스케일링 시나리오도 고려해주세요.

3.10 기본 원칙 - 가정과 한계 표기

❖사용 예시 3

- 기존 웹애플리케이션의 성능 최적화

현재 코드베이스에서 webapp 디렉토리에 있는 기존 웹 애플리케이션의 성능을 개선해주세요.

현재 상황 가정
- 페이지 로딩 시간: 평균 5초
- 동시 사용자: 200명
- 서버: AWS EC2 m5.large (1vCPU, 1GB RAM)
- 데이터베이스: MySQL 5.7, 단일 인스턴스
- 프론트엔드: jQuery 기반 레거시 코드
- 이미지: 평균 2MB, 압축되지 않음

개선 한계
- 코드 리팩토링: 20% 이하만 가능
- 다운타임: 주말 4시간만 허용
- 외부 서비스 도입: 최대 2개

목표
- 페이지 로딩 시간: 2초 이하
- 동시 사용자: 500명 지원
- 서버 응답 시간: 200ms 이하

현실적인 최적화 방안을 단계별로 제시해주세요.

3.10 기본 원칙 - 가정과 한계 표기

❖가정과 한계 표기의 장점

- 현실적 솔루션 도출

```
### 이상적 환경 가정 시
"무제한 리소스로 완벽한 솔루션 제시"

### 현실적 가정 포함 시
"제한된 예산과 시간 내에서 실현 가능한 솔루션"

### 결과
- 즉시 적용 가능한 코드
- 단계적 확장 계획 포함
- 위험 요소 사전 식별
```

- 팀 커뮤니케이션 향상

```
### 이해관계자별 관점
- 개발팀: 기술적 제약사항 명확화
- 기획팀: 현실적 기능 범위 설정
- 경영진: 예산 대비 효과 이해

### 명확한 소통
- "이 기능은 예산 한계로 제외"
- "사용자 증가 시 이 부분이 병목"
- "6개월 후 스케일링 필요"
```

3.10 기본 원칙 - 가정과 한계 표기

❖표준 가정과 한계 템플릿

```
# 프로젝트 가정 및 한계 템플릿

### 기술적 가정
- 개발 환경: [환경 명세]
- 사용자 규모: [예상 사용자 수]
- 데이터 규모: [예상 데이터 크기]
- 성능 요구사항: [응답시간, 처리량]

### 비즈니스 가정
- 예산: [총 예산 및 월별 운영비]
- 일정: [개발 기간 및 마일스톤]
- 인력: [개발팀 구성]
- 규제: [법적 요구사항]

### 기술적 한계
- 인프라: [서버 스펙, 네트워크 대역폭]
- 호환성: [지원 브라우저, 디바이스]
- 확장성: [최대 처리 용량]
- 보안: [보안 수준 제약]

### 비즈니스 제약
- 시간: [출시 데드라인, 유지보수 시간]
- 비용: [예산 제한, 운영비 상한]
- 조직: [인력 제약, 스킬 제약]
- 정책: [회사 정책, 업계 규제]

### 스케일링 시나리오
- 2배 성장 시: [필요한 변경사항]
- 10배 성장 시: [아키텍처 재설계 포인트]
- 예산 증액 시: [추가 구현 가능 기능]
- 기간 연장 시: [품질 개선 요소]
```

3.11 전략 - CoT(Chain of Thought)

❖AI가 최종 답변에 도달하기까지의 사고 과정을 단계적으로 보여주도록 유도하는 방식

❖장점 : 복잡한 문제 해결의 정확도가 대폭 향상됨

❖유형

- Zero-shot CoT : 별도의 예시 없이 "단계별로 생각해보자" 같은 트리거 구문만 사용
- Few-shot CoT : 예시와 함께 사고하는 단계를 보여준 후 과제를 제시하여 문제 해결을 유도

현재 코드베이스의 /front 디렉토리에 있는 React 앱의 렌더링 성능을 개선해주세요. 다음 단계를 적용합니다.

분석 과정:
1단계: 현재 성능 이슈 파악
- 모든 사용자 카드가 매번 리렌더링
- 메모이제이션 부재

2단계: 최적화 기법 적용
- React.memo로 컴포넌트 메모이제이션
- useMemo/useCallback 활용

3단계: 개선된 코드 작성
[최적화된 코드 제시]

4단계: 추가 고려사항
- 가상화 (react-window)
- 지연 로딩 등

3.12 전략 - ReAct(Reasoning-Acting)

❖모델이 생각(Reasoning)과 행동(Acting, 도구 호출/정보 검색)을 번갈아 수행하는 기법

❖복잡한 문제 해결(특히 외부 지식, API, 검색 결과 등이 필요한 경우)에 적합

- 인간이 복잡한 문제에 대한 자세한 지식이 없을 경우 유용한 방법

❖Think – Act – Observe(관찰,검토) – 계속 반복

- Think : 현재 상황에 대한 분석과 다음에 할 일
- Act : 구체적으로 수행하는 작업
- Observe : 작업 결과 분석
- 예시 : 앱 로그인 오류 디버깅 과정
  - Think 1 : (인간) 로그인 버튼 클릭 응답이 없다. 네트워크 요청부터 확인해보자.
  - Act 1 : (AI) Network 활동이 없음
  - Observe 1 : (인간, AI) API 요청이 전송이 없다는 뜻
  - Think 2 : (인간) 프론트엔드 로그인 버튼에 대한 이벤트 핸들러 문제일까?
  - Act 2 : (AI) 로그인 버튼 이벤트 핸들러 코드 검토,분석
  - Observe 2 : (인간, AI) 이벤트 핸들러 함수내에 stopPropagation()으로 인해서 이벤트 전파가 일어나지 않음

3.12 전략 - ReAct(Reasoning-Acting)

❖ReAct와 검수 루프의 비교 : 둘은 비슷해보이지만 차이가 있음

구분	ReAct	검수 루프
작업 방식	정보가 부족한 상태에서 행동을 통해 정보를 채워가면서 결과 또는 답을 완성해나감	먼저 결과 또는 답을 생성한 뒤, 그 결과 또는 결과를 검토, 비판하여 개선해나감. 개선의 대상은 결과 또는 프롬프트 둘다 가능
기본 구조	생각-행동-관찰	생성-평가-개선
반복의 목적	다음 행동을 결정하기 위해	기존 결과의 문제점을 개선하기 위해
외부 도구 사용	매우 중요함. 정보가 부족한 상태에서 시작하기 때문에 다양한 정보 획득 과정이 반드시 필요함	필수 아님
이전 결과 수정	부수적으로 수정하게 됨	결과를 수정하는 것이 핵심 활동
대표 사용 사례	디버깅, 새로운 정보 검색	코드 리뷰 후 코드 수정
AI 역할	정보 탐색자, 작업 수행자	작성자, 검수자

❖한마디로 요약하면

- ReAct : "다음에 무엇을 해야 문제를 해결할 수 있을까?"
- 검수루프 : "지금 만든 결과에서 무엇을 해야 더 좋아질까?"

3.13 전략 - 자기 일관성과 역할 기반 프롬프팅

❖자기 일관성 (Self-consistency)

- 하나의 문제에 대해 여러 번의 독립적인 추론 과정을 수행한 후, 가장 일관되게 나타나는 응답을 선택하는 방법.
- "다수결 투표" 방식과 유사한 접근법
- 정확도를 높이고 안정성 확보
- 복잡한 추론이 필요한 경우에 적합

❖역할 기반 프롬프팅 (Role Prompting)

- AI가 특정 전문가, 직업군, 또는 캐릭터의 역할을 맡아 그들의 지식, 경험, 사고방식을 바탕으로 응답하도록 유도하는 기법
- 핵심 요소
  - 전문성 정의: 구체적인 직업, 학위, 경력을 명시
  - 지식 범위: 알고 있을 만한 정보와 경험을 구체화
  - 행동 양식: 어떤 방식으로 문제에 접근하는지 설명

4. 템플릿

❖범용 템플릿

```
### ROLE
너는 {분야/직무} 전문가다. 대상사용자는 {타깃 사용자}이며,
톤은 {톤}

### TASK
목표: {한 줄}
성공조건: {정량/정성 기준}

### CONTEXT
- 배경 요약: {200~400자}
- 제약: {성능/보안/라이선스/환경}
- 참고: {요약 인용 또는 표준/정책 핵심조항}

### OUTPUT FORMAT
- 형식: {예: 유효한 JSON만}
- 스키마:
  {
    "title": "string",
    "steps": "string[]",
    "risks": "string[]"
  }
```

```
### INSTRUCTIONS - DO
- {해야 할 것 1}
- {해야 할 것 2}

### INSTRUCTIONS - DON'T
- {하지 말 것 1}
- {하지 말 것 2}

### EXAMPLES
입력: {간단 입력}
출력: {정답 예시}

### QUALITY RUBRIC
- 정확성/일관성/완결성/형식 준수 4점 척도
- 자기검수: 평가기준 미달 시 수정 후 다시 제출
```

5. 도메인별 예시

❖코드 생성 / 리팩토링

- 원칙 강화
  - 런타임/프레임워크/버전 명기 (예: Node 22 + Vite).
  - 입출력/에러/복잡도 제한, 성능 목표 명시.
  - 테스트 우선: 요구 테스트 케이스를 먼저 고정.
- 예시 프롬프트

```
### ROLE
너는 숙련된 TypeScript 엔지니어다.

### TASK
React 커스텀 훅을 현재 코드베이스의
src/utils/useDebounce.ts 파일에 작성해

#### CONTEXT
- 환경: React 18, Vite 5, TS
- 훅: useDebounce(value, delay)
- 제약: 클로저/타이핑 엄격, 메모리 누수 금지

### OUTPUT FORMAT
- 유효한 TS 코드만, 불필요한 설명 금지.
```

```
### DO
- jest용 간단 테스트 케이스 3개 포함
- 엣지케이스(null/undefined) 처리

### DON'T
- 외부 라이브러리 금지

### QUALITY RUBRIC
- TS 오류 0개, 테스트 통과, ESLint 기본 규칙 위반 0
```

5. 도메인별 예시

❖버그/디버깅

- 원칙 강화
  - 재현 절차, 기대결과 vs 실제결과, 로그/스택트레이스
  - 가설→검증계획→수정패치 순으로 출력.
- 예시 프롬프트

```
### TASK
버그 원인 파악 & 수정 패치 제안

### CONTEXT
- 재현: npm run dev 명령어로 개발용 웹서버 실행 후 /users 로 이동 시 에러 발생
- 로그: `TypeError: cannot read properties of undefined (reading 'map') ..... `
- 코드: src/components/Users.tsx

### OUTPUT FORMAT
{
  "root_cause": "...",
  "fix_patch": "```diff₩n...```",
  "tests": ["..."]
}

### DO : 가설 2개 이상 비교, 재현성 체크리스트 제공
### DON'T : 추측으로 단정하지 말 것
```

5. 도메인별 예시

❖요구사항 분석 프롬프트 예시

```
### ROLE
너는 숙련된 소프트웨어 요구사항 분석가다.

### TASK
아래 회의록에서 기능 요구사항과 비기능 요구사항을 식별하고,
우선순위(P1, P2, P3)와 품질 점수를 부여하며, 개선 권고를 포함한
요구사항 명세 초안을 작성하라.

### CONTEXT
- 입력: 고객 인터뷰 및 회의록
- 시스템 유형: 온라인 쇼핑몰 웹 애플리케이션
- 분석 기준: 명확성, 테스트 가능성, 추적 가능성(각 0~100 점수화)

### OUTPUT FORMAT
- JSON 형식으로만 출력하며, 아래 스키마를 준수
{
  "functional_requirements": [
    { "id": "FR-1", "description": "", "priority": "",
      "quality_score": 0, "improvement": "" }
  ],
  "non_functional_requirements": [
    { "id": "NFR-1", "description": "", "priority": "",
      "quality_score": 0, "improvement": "" }
  ]
}
```

```
### DO
- 기능/비기능 요구사항 분리
- 우선순위와 품질 점수 산정
- 개선 권고 1개 이상 포함

### DON'T
- 요구사항 해석 시 모호한 표현 사용 금지
- 출력 형식 벗어난 서술 금지

### QUALITY RUBRIC
- 모든 요구사항에 우선순위·점수·개선안 포함
- JSON 스키마 100% 준수
- 모호한 표현(예: "빠른", "편리한") 미사용
```

5. 도메인별 예시

❖설계 프롬프트 예시

```
### ROLE
너는 숙련된 소프트웨어 요구사항 분석가다.

### TASK
아래 회의록에서 기능 요구사항과 비기능 요구사항을 식별하고, 우선순위(P1, P2, P3)와 품질 점수를 부여하며, 개선 권고를 포함한 요구사항 명세 초안을 작성하라.

### CONTEXT
- 입력: 고객 인터뷰 및 회의록
- 시스템 유형: 온라인 쇼핑몰 웹 애플리케이션
- 분석 기준: 명확성, 테스트 가능성, 추적 가능성(각 0~100 점수화)

### OUTPUT FORMAT
- JSON 형식으로만 출력하며, 아래 스키마를 준수
{
  "functional_requirements": [
    { "id": "FR-1", "description": "", "priority": "",
      "quality_score": 0, "improvement": "" }
  ],
  "non_functional_requirements": [
    { "id": "NFR-1", "description": "", "priority": "",
      "quality_score": 0, "improvement": "" }
  ]
}
```

6. 안티 패턴

❖컨텍스트 관련 안티 패턴

- 모호한 요구사항
  - 구체적인 기술 스택, 프레임워크, 스타일 가이드를 명시하지 않고 "웹사이트 만들어줘"처럼 애매하게 요청
  - 수정 예시) "React와 TypeScript로 반응형 대시보드를 만들어줘, Material-UI 사용하고 다크모드 지원해야 해"
- 컨텍스트 누락
  - 기존 코드베이스의 구조, 네이밍 컨벤션, 아키텍처 패턴을 설명하지 않는 것
  - AI가 프로젝트의 전체적인 맥락을 이해할 수 있도록 관련 정보를 충분히 제공해야 함.

❖스코프(Scope) 관리 안티패턴

- 거대한 일괄 요청
  - 한 번에 전체 애플리케이션을 요청하는 것은 바람직하지 않음.
  - 컴포넌트나 기능 단위로 나누어 점진적으로 개발하는 것이 좋음
- 무분별한 기능 확장
  - 기본 기능이 완성되기 전에 계속해서 새로운 기능을 추가 요청하는 것은 바람직하지 않음
  - 핵심 기능을 먼저 완성하고 단계적으로 확장하는 것이 바람직함

6. 안티 패턴

❖커뮤니케이션 안티패턴

- 결과 검증 생략
  - AI가 생성한 코드를 바로 사용하지 말고 반드시 테스트하고 검토해야 함.
  - 오류나 개선점을 발견하면 구체적인 피드백을 제공하여 다음 반복에서 더 나은 결과를 얻을 수 있도록 해야 함
- 일방적인 지시
  - 단순히 명령만 내리지 말고 AI와 대화하듯 상호작용하는 것이 바람직함.
  - "이 방법 말고 다른 접근법은 어때?" 같은 질문으로 다양한 옵션을 유도하는 것이 좋음

❖기술적 안티패턴

- 모범 사례 무시
  - "빨리 돌아가기만 하면 돼"라는 식으로 요청하면 품질이 낮은 코드가 생성됨.
  - 보안, 성능, 유지보수성을 고려한 코드를 요청해야 함
- 에러 처리 생략
  - 에러 처리, 예외 상황, 엣지 케이스에 대한 고려 없이 해피 패스만 구현을 요청하는 것
  - 실제 프로덕션에서 발생할 수 있는 다양한 상황을 고려해야 함.

6. 안티 패턴

❖프로세스 안티패턴

- 문서화 무시
  - 코드만 요청하고 주석이나 README, API 문서 작성 수행하지 않는 것
  - 나중에 코드를 이해하고 유지보수하기 위해서는 적절한 문서화가 필수
- 버전 관리 무시
  - 변경사항을 추적하지 않고 계속해서 코드를 수정 요청하면 이전 버전으로 돌아가기 어려워짐
  - 중요한 변경점에서는 백업을 유지하는 것이 바람직함
- 테스트 코드 생략
  - 기능 구현만 요청하고 테스트 코드 작성을 누락하는 것.
  - 단위 테스트, 통합 테스트를 함께 요청하여 코드의 안정성을 확보해야 함.

7. 정리

❖ 정의/목적
- 프롬프트 엔지니어링은 LLM이 원하는 결과를 내도록 입력을 설계·최적화해 정확성·효율·일관성을 높이고 자동화를 돕는 과정.

❖ 구성요소
- 목표(Task), 입력(Context), 출력 형식, 지시어(Do/Don't), 평가기준(Rubric), 역할/톤, 예시를 이용한 구조화.

❖ 핵심 원칙
- 명확성·입력 분리(구분자)·출력 고정(형식 락)·단계적 분해·최소 충분 컨텍스트·긍/부정 지시 병행·예시 기반·검수 루프·도구 사용 선언·가정/한계 표기.

❖ 추론·행동 기법
- CoT, ReAct, 자기 일관성, 역할 기반 프롬프팅으로 복잡 문제 해결력과 안정성 강화.

❖ 템플릿/도메인 예시
- 범용 ROLE/TASK/CONTEXT/OUTPUT/INSTRUCTIONS/RUBRIC 템플릿과 코드 생성·디버깅·요구분석·설계 등 상황별 예시 제공.

❖ 안티패턴/품질보증
- 모호한 요구·컨텍스트 누락·일괄요청·검증/테스트/문서화·버전관리 무시를 금지하고, 체크리스트 기반 검수 루프로 품질을 체계적으로 개선.